

CSCI 447 — Machine Learning

Project 1

Assigned: August 26, 2026
Project Due: September 18, 2026

1 Introduction

The purpose of this assignment is to give you some hands-on experience implementing a nonparametric algorithm to perform classification and regression. Basically, you will be implementing a k -nearest neighbor classifier and regressor. Throughout this course you will use six datasets that you will download from the UCI Machine Learning Repository¹. Details about these datasets can be found toward the end of these instructions. Although these datasets all differ, you should plan to build a general set of tools that can handle any dataset. Test your functions well; your future self will thank you.

2 Null Model Predictors

Although the purpose of this first lab is to experiment with nonparametric, nearest neighbor machine learning, you need a way to test your approach against some kind of baseline approach. For this purpose, you will create very simple “Null Model” predictors. These are very naïve algorithms that will simply return the plurality (most common) class label in classification tasks, and the average of the outputs for regression tasks. Because null models do not consider any of the feature values in its decision, you should not expect high accuracy nor low error results. But these simple models will enable you to test whatever process or pipeline you put in place and serve as a baseline for comparison. In future assignments, you may even find you can adapt them for use as a part of the more complicated algorithm being implemented.

3 Nonparametric Algorithms

In completing this assignment, be very careful with how the features are handled. Nearest neighbor methods work best with numeric features, so some care will need to be taken to handle categorical (i.e., discrete) features. The typical approach to handling numeric features is to first normalize them (either using min-max normalization or z-score normalization) and then, when finding neighbors, use a Minkowski metric of the form

$$D_{\text{num}}(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^d |x_k - y_k|^p \right)^{1/p}$$

where classic values for p are $p = 1$ (Manhattan distance), $p = 2$ (Euclidean distance), or $p = \infty$ (max-coordinate distance).

One way of handling categorical features is with the Value Difference Metric (VDM). Note that you are not required to use VDM, but you may find this to be a useful method. Note further that VDM only works with classification. If you have categorical features in regression, you will need to apply a different approach (such as one hot coding with Hamming distance, which would also work for regression).

For VDM, let $f(\mathbf{x})$ be a feature of data instance $\mathbf{x}$, where $f(\mathbf{x}) \in \{v_1, \dots, v_k\}$. Let $C_i = \#\{f(\mathbf{x}) = v_i\}$ (i.e., the number of times feature f takes on value v_i in the training set) and $C_{i,a} = \#\{f(\mathbf{x}) = v_i \wedge \text{class} = a\}$ (i.e., the number of examples in the training set belong to class a that also have feature value v_i for feature f). We then define the “distance” between feature value v_i and feature value v_j as

$$\delta(v_i, v_j) = \sum_{a=1}^{\text{num classes}} \left| \frac{C_{i,a}}{C_i} - \frac{C_{j,a}}{C_j} \right|^p.$$

¹Note that all of the data sets are also available in the content area within Canvas

where p is the exponent that defines the associated norm similar to an L_p norm. Typically, you will set $p = 1$ or $p = 2$ (tune this). Then the distance between two examples $\mathbf{x}$ and $\mathbf{y}$ is

$$D_{\text{cat}}(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^d \delta(\mathbf{x}_k, \mathbf{y}_k) \right)^{1/p}.$$

A question that is often asked is with respect to how to combine numerical and categorical distance. To address this, we assume that we do not take the p th root. We justify this by observing that the p th root does not change the ordering of the neighbors, so we can still pick the k nearest correctly. In this case, we calculate distances based on the relevant feature type while using the same value of p for each and then just add the two distances together:

$$D(\mathbf{x}, \mathbf{y}) = D_{\text{num}}(\mathbf{x}, \mathbf{y}) + D_{\text{cat}}(\mathbf{x}, \mathbf{y})$$

4 List of Data Sets

Use the following information on all of the assignments except the Reinforcement Learning assignment. Note that you are expected to read and understand all of the associated “.names” files for these data sets.

1. Breast Cancer [Classification]

Overview: This data describes characteristics of cell nuclei present in benign and malignant tumors.

Predictor: Diagnosis: M or B

Source: University of Wisconsin, 1993

URL: <https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+%28Original%29>

Notes: The classes are indicated with the values “2” and “4”, but these are just two discrete classes and should be treated as such. The features are all values in the range $[1, 10]$ and should be treated either as numeric or ordinal features. The Sample code number should be discarded. Missing values may be imputed, or the examples with missing features may be dropped from the data set.

2. Car Evaluation [Classification]

Overview: The data is on evaluations of car acceptability based on price, comfort, and technical specifications.

Predictor: CAR: unacc, acc, good, vgood

Source: Jozef Stefan Institute, Yugoslavia (Slovenia), 1988

URL: <https://archive.ics.uci.edu/ml/datasets/Car+Evaluation>

Notes: All of the features should be treated as if they are nominal even though, technically, they are ordinal. This is a four-class problem.

3. Congressional Vote [Classification]

Overview: This data set includes votes for each of the U.S. House of Representatives Congressmen on the 16 key votes identified by the Congressional Quarterly Almanac.

Predictor: Class: democrat, republican

Source: University of California, Irvine, 1987

URL: <https://archive.ics.uci.edu/ml/datasets/Congressional+Voting+Records>

Notes: Be careful with this data set since “?” does not indicate a missing attribute value. It actually means “abstain.” This is a two-class problem.

4. Abalone [Regression]

Overview: The data describes the physical measurements of abalone and the associated age.

Predictor: Rings (`int`)

Source: Marine Research Laboratories, Tasmania, 1995

URL: <https://archive.ics.uci.edu/ml/datasets/Abalone>

Notes: Observe that, even though the “names” file lists 29 classes, this is not to be treated like a classification problem. You are trying to predict (regress) the number of rings. All of the features are real-valued except for sex. You may choose between one-hot coding this feature, or discarding it.

5. Computer Hardware [Regression]

Overview: The data describes relative CPU performance described by features such as cycle time, memory size, etc.

Predictor: PRP (`int`)

Source: Tel Aviv University, Israel, 1987

URL: <https://archive.ics.uci.edu/ml/datasets/Computer+Hardware>

Notes: The features for vendor name and model name are not useful for regression, so these features should be discarded. All of the remaining features are real-valued (numeric). The estimated relative performance ERP values were estimated by the authors using a linear regression method. This **cannot** be used as a feature. You should remove it from the feature set, but save it elsewhere. It might be interesting to test your regressors against this value as part of your experimental results. Even though a class distribution is provided, this is not a classification problem and should not be treated as such.

6. Forest Fires [Regression]

Overview: This is a difficult regression task, where the aim is to predict the burned area of forest fires by using meteorological and other data.

Predictor: area (`float`)

Source: University of Minho, Portugal, 2007

URL: <https://archive.ics.uci.edu/ml/datasets/Forest+Fires>

Notes: The output **area** is very skewed toward 0.0. The authors recommend a log transform of the form $\hat{r}(\mathbf{x}) = \ln(r(\mathbf{x}) + 1)$, where $r(\mathbf{x})$ is the target value associated with example $\mathbf{x}$. The spatial coordinates should be treated as numeric features. The month and day features are ordinal and should use label encoding; however, be careful in computing distance since these are “cycle” features (e.g., in the absence of date information, the distance between December and January is only 1.0, not 11.0).

5 Requirements

For this project, the following steps are required:

- Download the six (6) data sets from the UCI Machine Learning repository. You can find this repository at <http://archive.ics.uci.edu/ml/>. All of the specific URLs are also provided below. All of the data sets are also available in Canvas itself.
- Implement the null models for classification and regression.
- Implement k -nearest neighbor and be prepared to find the best k value for your experiments. To do this, you must tune k and explain in your report how you did the tuning.
- Implement both edited or condensed k -nearest neighbor. See above with respect to tuning k ; however use $k = 1$ during the editing/condensing process. Only use the tuned k when doing a prediction using your reduced data set. On the regression problems, you should define an error threshold ϵ to determine if a prediction is correct or not. This ϵ will also need to be tuned.

- For classification, employ a plurality vote to determine the class (i.e., return the class with the highest number of votes among the neighbors; if there's a tie, choose randomly). For regression, apply a Gaussian (radial basis function) kernel to make your prediction. In particular, you should use the kernel $K(\mathbf{x}, \mathbf{x}_q) = \exp(-\gamma \|\mathbf{x} - \mathbf{x}_q\|_2)$, where $\|\cdot\|_2$ indicates the L2 norm (i.e. Euclidean distance in this case). You will need to tune the bandwidth $\gamma = 1/\sigma$.
- Run all of your algorithms on each of your chosen data sets. These runs shall be done with 5×2 cross-validation so you can compare your results statistically. You should use classification error to evaluate your classifier and mean squared error to evaluate your regressor. Other metrics are permitted, but you must explain and justify your choice if you don't use the above.
- Write a lab report that incorporates the following elements, summarizing the results of your experiments. Your report is required to be formatted using the JMLR format. You can find templates for this format at <http://www.jmlr.org/format/format.html>.

All project reports must be no more than 15 pages using JMLR format and include the following:

- Title and authors
 - Problem statement with at least one testable hypothesis
 - Experimental approach and explanation of the project design
 - Explanation of the algorithms and how they were implemented (without showing code)
 - Experimental results (tables and/or graphs with explanation)
 - Analysis and discussion of results
 - Two appendices (do not count in page limit):
 - * Main lessons learned
 - * Delineation of who did what on the project
 - Conclusion
 - References (if using any not provided in course content)
- Submit a PDF of your report to the “P1 Paper” portion of the assignment. Canvas is set up to accept PDF files only. Your report will be checked using the TurnItIn plagiarism checking system. The paper must be formatted per JMLR formatting requirements. All math should be included using the math editing capabilities in LaTeX. Figures need to be high quality. Do not use screenshots.